Algorithms and Analysis Report

COSC2123/3119

Graph Algorithms in Action: Modelling a Disease Outbreak

Your Name & Student Number Here

Assessment Type	Individual assignment. Submit online via GitHub.
Due Date	Week 11, Thursday May 21, 13:00 (1pm). A late penalty will apply to assessments submitted after 13:00.
Marks	30

1 – Learning Outcomes

This assessment relates to four learning outcomes of the course which are:

- CLO 1: Compare, contrast, and apply the key algorithmic design paradigms: brute force, divide and conquer, decrease and conquer, transform and conquer, greedy, dynamic programming and iterative improvement;
- CLO 2: Compare, contrast, and apply key data structures: trees, lists, stacks, queues, hash tables and graph representations;
- CLO 3: Define, compare, analyse, and solve general algorithmic problem types: sorting, searching, graphs and geometric;
- CLO 4: Theoretically compare and analyse the time complexities of algorithms and data structures; and
- CLO 5: Implement, empirically compare, and apply fundamental algorithms and data structures to real-world problems.

2 – Overview

Across multiple tasks in this assignment, you will model a disease outbreak in the city of Metropolis as a weighted graph to capture residents' contacts, implement and compare algorithms for estimating infection risk through the population, and design a dynamic programming strategy to allocate scarce antiviral treatment as effectively as possible. Some components ask you to critically assess your solutions through both theoretical analysis and controlled empirical experiments, encouraging reflection on the relationship between algorithm design and real-world performance.

I certify that this is all my own original work. If I took any parts from elsewhere, then they were non-essential parts of the assignment, and they are clearly attributed in my submission. I will show I agree to this honor code by typing "Yes": YOUR ANSWER HERE

3 – Motivation

The T-Virus has broken out in the city of Metropolis. As the newly appointed Regional Health Director, you have been handed a dossier containing everything the epidemiologists know: a map of every known contact between residents, the probability that each contact transmits the virus in a single day, and the vulnerability of each resident. The clock is ticking — every day the virus spreads further, and your antiviral stockpile is limited.

Your job has four parts:

1. **Build the contact network** (Task A) — represent the city's contact relationships as a graph, efficiently enough to run algorithms on in real time (implement `graph/adjacency_matrix.py`).
2. **Estimate infection risk** (Task B) — compute the probability that each resident becomes infected over the coming T days, identifying the most at-risk residents before the outbreak grows (implement `transmission/task_b.py` and write report).
3. **Analyse your solution** (Task C) — theoretically and empirically compare your algorithm against the provided baseline across different network structures (write report on analysis of task A and B).
4. **Allocate antiviral treatment** (Task D) — decide who receives the scarce supply of doses, maximising the benefit to the population within the limits of your stockpile (implement `treatment/task_b.py` and write report).

Figure 1 shows the contact network for sub-population in Metropolis. Each resident is a node, and each contact relationship is an edge labelled with the daily probability of transmission. Patient zero V_0 is already infected — the question is how far the virus will spread, and who we should protect first.

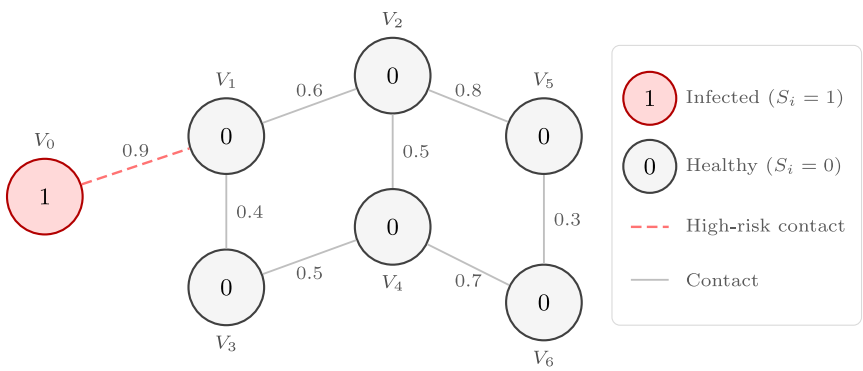

Figure 1: The contact network $G = (V, E)$ for Metropolis. V_0 is patient zero. Edge labels give the daily transmission probability $w_{ij} \in (0, 1]$. Dashed red edges originate from an infected resident.

By vaccinating even a small number of strategically chosen residents, we can sever the most dangerous transmission pathways through the network. Figure 2 shows an example: vaccinating V_1 — the single resident through whom all transmission from V_0 must pass — protects every downstream resident at once.

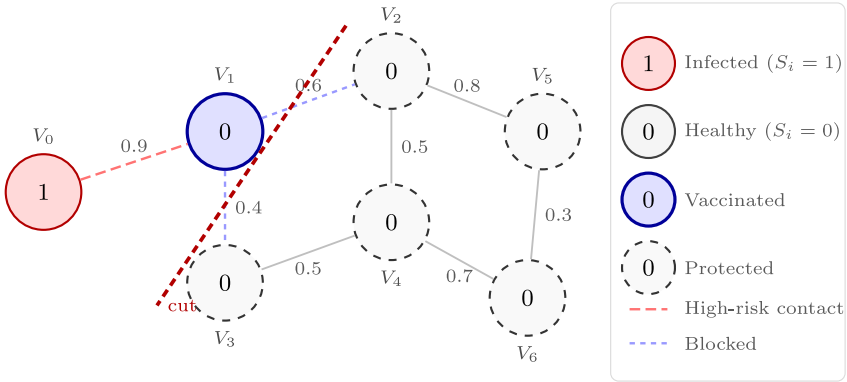

Figure 2: Vaccinating V_1 — the single bottleneck through whom all transmission from V_0 must pass — severs every onward pathway at once, protecting V_2 through V_6 entirely. This illustrates the core challenge of Task D: identifying which residents to vaccinate in order to maximise protection across the network given a limited supply of doses.

3.1 – Mathematical Background

We model the city of Metropolis as a weighted undirected graph $G = (V, E)$, where:

- each vertex $V_i \in V$ represents a resident, with infection state $S_i \in \{0, 1\}$ where $S_i = 0$ is healthy and $S_i = 1$ is infected;
- each edge $(V_i, V_j) \in E$ represents a contact relationship between two residents; and
- each edge carries a weight $w_{ij} \in (0, 1]$, the probability that an infected resident V_i transmits the virus to a healthy neighbour V_j in a single day.

Each resident V_i also has two personal attributes relevant to the vaccine allocation in Task D:

- a **dosage requirement** $c_i \in Z^+$, the number of antiviral doses required to vaccinate resident V_i; and
- a **vulnerability score** $\phi_i \in (0, 1]$, reflecting how severely resident V_i would be harmed if infected.

Once infected, a resident remains infectious forever. Over a planning horizon of T days, the probability that an infected resident V_i infects neighbour V_j at least once is:

$$p_{ij} = 1 - (1 - w_{ij})^T$$

As $T \to \infty$, $p_{ij} \to 1$ for any $w_{ij} > 0$ — given enough time, every resident reachable from patient zero will eventually be infected. This makes early intervention critical. The benefit of vaccinating resident V_i is therefore a function of how likely they are to be infected, how severely they would be harmed, and how much onward transmission they would cause — all of which are computed from G, $r_{i,T}$, and ϕ_i in Tasks B and D.

Task B: Estimating Infection Risk Over Time (7 marks, 1 page)

Pseudo-code

The Monte Carlo baseline (Algorithm 1) is provided as a style guide. Write your pseudo-code for the dynamic programming solution below in the same style.

Algorithm 1 MonteCarlo(G, s, T, X)

Require: Graph $G = (V, E)$; source s; horizon T; simulations X
Ensure: Estimated risk table where $\texttt{table}[t][i] \approx r_{i,t}$

```
for each day t = 1 to T do
    for each simulation x = 1 to X do
        Restart from scratch — only s is infected
        for each day d = 1 to t do
            for each uninfected resident V_i do
                for each infected neighbour V_j of V_i do
                    Infect V_i with probability w_ij
    table[t][i] ← fraction of simulations where V_i infected
return table
```

Limitations of the Recurrence

(a) *Your answer to part (a) here.*

(b) *Your answer to part (b) here, with reference to Figure 3.*

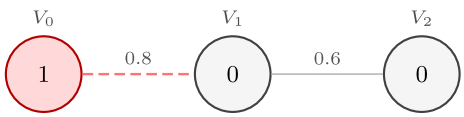

Figure 3: This is Figure 3

(c) *Your answer to part (c) here.*

Task C: Analysis of Infection Risk Algorithms (8 marks, 2 pages)

Algorithm Complexity Analysis

Your complexity analysis of all four combinations here.

Empirical Design

Describe which variables vary, which you fix, and why.

Empirical Analysis

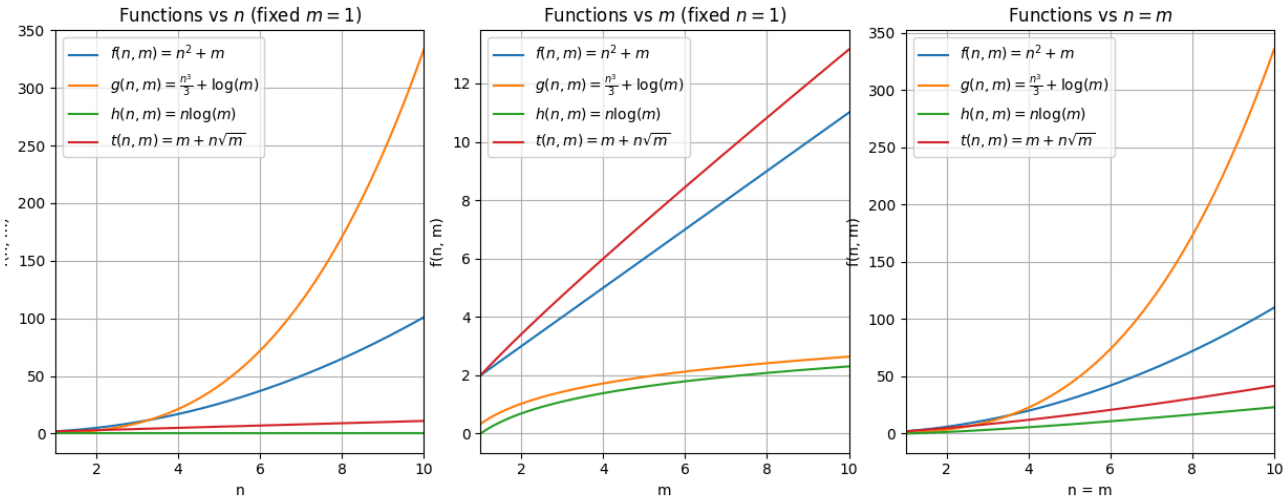

Figure 4: Runtime comparison of the four algorithm and representation combinations. *Replace this with your own plot and caption.*

Reflection

Your reflection here, including a recommendation for Metropolis and a discussion of how your answer would change if transmission rates and connections updated daily.

Task D: Antiviral Allocation (10 marks, 2 pages)

Algorithm Design

Describe the algorithm you designed

Complexity Analysis

Justify the time complexity of your solution. Explain where each factor comes from.

Extension 1: Triage

Describe how you would modify your solution to handle K vulnerability tiers. State the complexity of your modified solution. Include a small numerical counterexample showing that ignoring vulnerability can lead to a highly vulnerable resident being passed over.

Extension 2: Interdependent Vaccinations

Explain why the independence assumption $b_i = r_{i,T}$ is necessary for your solution to be valid. Construct a small example showing that ignoring interdependence leads to a sub-optimal outcome. Explain why the true optimal problem under interdependence is significantly harder to solve.